Gradient Descent Methods (Lec8.pdf)

Ioannis Akrotirianakis

Todays topics & Logistics

- Gradient descent methods
  - Descent directions
  - Step sizes
  - Convergence
  - Application to quadratic optimization

- **HW4 → due on March 22 @ 23:59**
- **Midterm Exam**
  - **Booklets reunite with the students**
  - **Anything you want to discuss please reach out!**

The General Iterative Formula

- We have seen several **characterizations for local optimality** for the unconstrained optimization problem $\min_{x \in R^n} \; f(x)$ *(f is twice continuously differentiable)*
  - FONC, SONC, SOSC
- Essentially, we have to find **candidate points**, in a **computationally efficient manner**
  - Need to solve the system of nonlinear equations: $\nabla f(x) = 0$ **It may contain 100,000s of decision variables**
  - We solve it **iteratively** $x_{k+1} = x_k + \alpha_k d_k$

The General Iterative Formula

- We solve it **iteratively** $x_{k+1} = x_k + \alpha_k d_k$
  - k is the iteration counter (or you think of it as the *time → connection to Dynamical Systems*)
  - $x_k \in R^n$ is the current iteration (carries all the information acquired so far)
  - $x_{k+1} \in R^n$ is the next iteration (we want it to be closer to x^*)
  - $d_k \in R^n$ is the direction of movement at iteration k
  - $\alpha_k > 0$ is the step size we take
- **Main goal:** make the sequence $\{f(x_k): k = 0,1,2, \dots\}$ **decrease sufficiently** at every iteration

The General Iterative Formula

- **Main goal:** make the sequence $\{f(x_k): k = 0,1,2, \dots\}$ **decrease sufficiently** at every iteration k

- **Three main computational tasks:**

$$x_{k+1} = x_k + \alpha_k d_k$$

1. How to compute the **direction of movement** d_k
   - Think of the Gradient of f $(\nabla f(x_k))$ → what do we know already??
2. How to compute the **step size** α_k (or learning rate)
   - Think of one-dimensional optimization
3. When to **stop**
   - Have the iterates x_k approached the optimal solution x^* ??

- The **initial point** x_0
  - It is usually selected at random
  - The closer to the optimal solution the better

Descent directions

There are many directions we can follow…

But what is a good direction of movement when we want to go downhill ??

Definition: A direction $d \in R^n$ is called **descent direction** for the function $f: R^n \to R$ at a point $x \in R^n$ iff there exists $\bar{\alpha} > 0$ such that

$$\boxed{f(x + \alpha d) < f(x), \forall \alpha \in (0, \bar{\alpha})}$$

In plain English: there is a small (nonnegative) **step size** α that we can move along the **direction** d and **be certain that the function f will decrease**.

Descent directions

Lemma 1: Consider the continuously differentiable function $f: R^n \to R$ and a point $x \in R^n$. The direction $-\boldsymbol{\nabla f(x)}$ is the direction of maximum rate of decrease at x

Lemma 2: Consider the continuously differentiable function $f: R^n \to R$ and a point $x \in R^n$. Any direction $d \in R^n$ satisfying:

$$\boxed{d^T \nabla f(x) < 0}$$

is a descent direction of f at the point $x \in R^n$

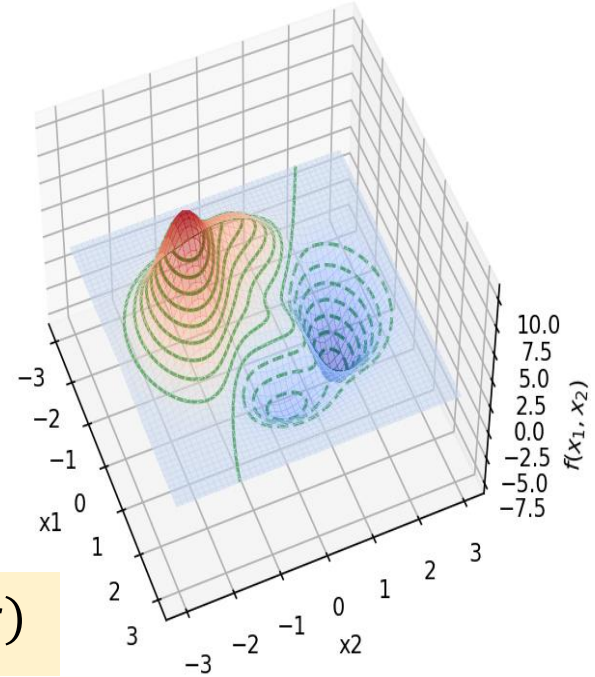
$$d^T \nabla f(x) = ||d||\, ||\nabla f(x)|| \cos(\theta)$$

d is descent direction iff $\theta \in [\frac{\pi}{2}, \frac{3\pi}{2}]$

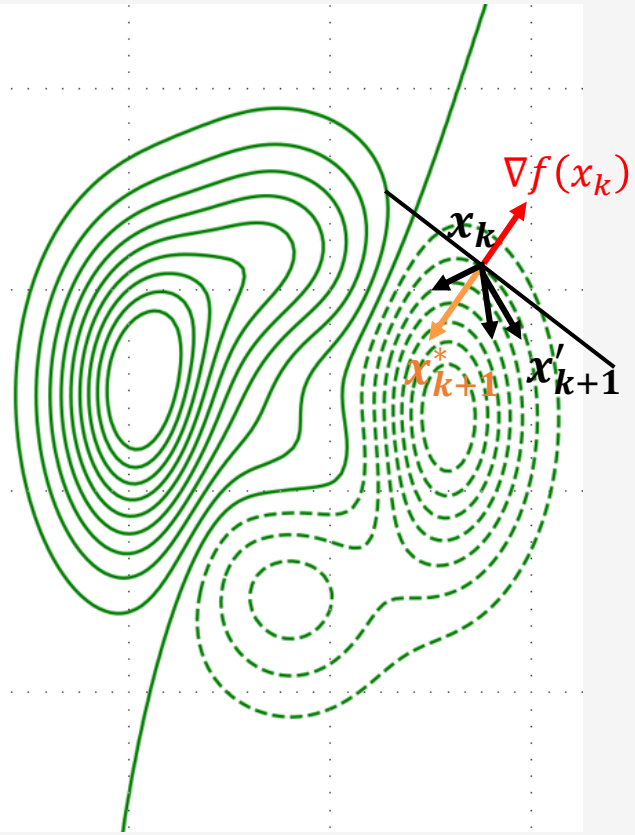
Corollary: $d = -\nabla f(x)$ is descent direction

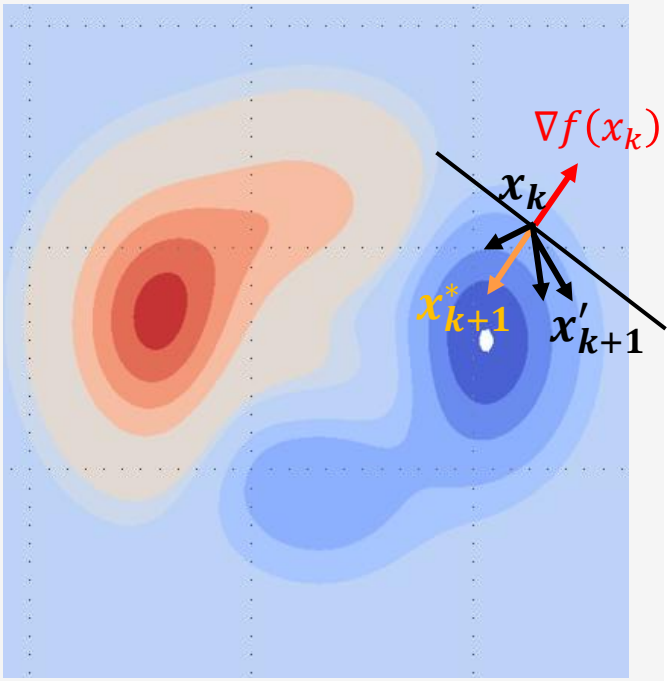
Descent directions – intuition & visualizations

$f(x_1, x_2) = 3(1-x_1)^2 exp(-x_1^2 - (x_2+1)^2) + 10*(0.2x_1 - x_1^3 - x_2^5)*exp(-(x_1^2+x_2^2))$



$$\min_{x \in R^n} \; f(x)$$





$$d^T \nabla f(x) = ||d||\,||\nabla f(x)|| \cos(\theta)$$

d is descent direction iff $\theta \in [\frac{\pi}{2}, \frac{3\pi}{2}]$

$\longrightarrow$ $\nabla f(x)$

$\longrightarrow$ $-\nabla f(x)$

$\longrightarrow$ d

$$x^*_{k+1} = x_k + \alpha(-\nabla f(x_k))$$

$$x'_{k+1} = x_k + \alpha d$$

for some $\alpha > 0$

Local minima and maxima in nature…

Can you see the **Descent Directions** ?

Random screen saver on my laptop – thanks to Microsoft

The Gradient Descent algorithm (pseudocode)

- **Step 0:** Select an initial point x_0, set $k = 0$, and choose a small $\epsilon > 0$
- **Step 1:** Calculate a descent direction $d_k = -\nabla f(x_k)$;

  (application of FONC)
- **STEP 2:** If $||\nabla f(x_k)|| \leq \epsilon$ Then STOP → x_k is a candidate optimal solution
- **Step 3:** Choose step size α_k
- **Step 4:** Find next iterate: $x_{k+1} = x_k + \alpha_k d_k$
- **Step 5:** Set $k = k + 1$ and GOTO **Step 1**

Proofs

Proof of Lemma 1: we need to show: $-\nabla f(x)$ is the direction of maximum rate of decrease at x

Consider the <u>one-variable function</u> $g(\alpha) = f(x + \alpha \frac{d}{||d||})$ for some direction $d \in R^n$

Proofs

Proof of Lemma 1: we need to show: $-\nabla f(x)$ is the direction of maximum rate of decrease at x

Consider the <u>one-variable function</u> $g(\alpha) = f(x + \alpha \frac{d}{||d||})$ for some direction $d \in R^n$

chain rule

Let's find: $g'(a) = \nabla f\left(x + \alpha \frac{d}{||d||}\right)^T \left(x + \alpha \frac{d}{||d||}\right)' = \frac{1}{||\mathrm{d}||} \nabla f\left(x + \alpha \frac{d}{||d||}\right)^T d$

Proofs

Proof of Lemma 1: we need to show: $-\nabla f(x)$ is the direction of maximum rate of decrease at x

Consider the <u>one-variable function</u> $g(\alpha) = f(x + \alpha \frac{d}{||d||})$ for some direction $d \in R^n$

chain rule

Let's find: $g'(a) = \nabla f\left(x + \alpha \frac{d}{||d||}\right)^T \left(x + \alpha \frac{d}{||d||}\right)' = \frac{1}{||\mathrm{d}||} \nabla f\left(x + \alpha \frac{d}{||d||}\right)^T d$

The rate of change of $\boldsymbol{f}$ at x along the direction $\boldsymbol{d}$ is given by $\boldsymbol{g}'(\mathbf{0})$

Hence if $\alpha = 0$ we get $g'(0) = \frac{1}{||\mathrm{d}||} \nabla f(x)^T d$

Using the Cauchy-Schwartz inequality: $\overbrace{-\frac{1}{||\mathrm{d}||} \left|\left|\nabla f(x)\right|\right| \left|\left|d\right|\right|}^{\textbf{LB}} \leq \frac{1}{||\mathrm{d}||} \nabla f(x)^T d \leq \overbrace{\frac{1}{||\mathrm{d}||} ||\nabla f(x)|| \, ||d||}^{\textbf{UB}}$

Proofs

Using the Cauchy-Schwartz inequality: $\overbrace{-\left|\left|\nabla f(x)\right|\right|}^{\textbf{LB}} \leq \frac{1}{||\mathrm{d}||}\nabla f(x)^T d \leq \overbrace{||\nabla f(x)||}^{\textbf{UB}}$

We reach the **LB** (i.e., the maximum rate of decrease) if we take $d = -\nabla f(x)$

(when we **minimize**)

$$\frac{1}{||\mathrm{d}||}\nabla f(x)^T d = \frac{1}{||-\nabla f(x)||}\nabla f(x)^T\left(-\nabla f(x)\right) = -\frac{1}{||\nabla f(x)||}\left|\left|\nabla f(x)\right|\right|^2 = -\left|\left|\nabla f(x)\right|\right| < 0 \quad \blacksquare$$

Remark: We reach the **UB** (i.e., the maximum rate of increase) if we take: $d = \nabla f(x)$

(when we **maximize**)

$$\frac{1}{||\mathrm{d}||}\nabla f(x)^T d = \frac{1}{||\nabla f(x)||}\nabla f(x)^T\left(\nabla f(x)\right) = \frac{1}{||\nabla f(x)||}\left|\left|\nabla f(x)\right|\right|^2 = ||\nabla f(x)||$$

Proofs

Proof of Lemma 2: we need to show: any direction $d \in R^n$ such that $\mathbf{d}^{\mathrm{T}}\nabla f(x) < \mathbf{0}$ is a **descent direction** of f at $x \in R^n$.

Recall Taylor's 1st order expansion of f **at** $\boldsymbol{x}$

$$f(\boldsymbol{x} + \boldsymbol{\alpha d}) = f(x) + \nabla f(x)^T(\boldsymbol{x} + \boldsymbol{\alpha d} - x) + o(\alpha) = f(x) + \alpha \nabla f(x)^T d + o(\alpha) \qquad \text{(Eq.1)}$$

with $\quad lim_{\alpha \to 0} \frac{|o(\alpha)|}{\alpha} = 0,$

Using our assumption $\mathrm{d}^{\mathrm{T}}\nabla f(x) < 0$, we can transform (Eq.1) as follows:

$f(x + \alpha d) = f(x) + \alpha \nabla f(x)^T d + o(\alpha) \Rightarrow f(x + \alpha d) < f(x)$ and hence d is descent direction ■

Corollary: if $\boldsymbol{d} = -\boldsymbol{\nabla f(x)}$ then we have: $\mathrm{d}^{\mathrm{T}}\nabla f(x) = (-\nabla f(x))^{\mathrm{T}}\nabla f(x) = -\left|\left|\nabla f(x)\right|\right|^2 < 0$ and hence $-\boldsymbol{\nabla f(x)}$ **is descent**

Descent directions – multiplication with a PD matrix

Lemma 3: Consider a symmetric **PD matrix** $\boldsymbol{B} \in \boldsymbol{R}^{\boldsymbol{n}\times\boldsymbol{n}}$ $(\boldsymbol{B} \succ \boldsymbol{0})$ and a continuously differentiable function $f: R^n \to R$. For any point $x \in R^n$ with $\nabla f(x) \neq 0$, the direction $\widehat{\boldsymbol{d}} = -\boldsymbol{B}\boldsymbol{\nabla}\boldsymbol{f}(\boldsymbol{x})$ is a descent direction

Proof: we simply need to apply Lemma 2.

$$\hat{d}^T \nabla f(x) =$$

Descent directions – multiplication with a PD matrix

Lemma 3: Consider a symmetric **PD matrix** $\boldsymbol{B} \in \boldsymbol{R}^{\boldsymbol{n}\times\boldsymbol{n}}$ $(\boldsymbol{B} \succ \boldsymbol{0})$ and a continuously differentiable function $f: R^n \to R$. For any point $x \in R^n$ with $\nabla f(x) \neq 0$, the direction $\widehat{\boldsymbol{d}} = -\boldsymbol{B}\boldsymbol{\nabla}\boldsymbol{f}(\boldsymbol{x})$ is a descent direction

Proof: we simply need to apply Lemma 2.

$$\hat{d}^T \nabla f(x) = -(B\nabla f(x))^T \nabla f(x) = -\nabla f(x)^T B \nabla f(x) < 0$$

Hence $\hat{d}^T \nabla f(x) < 0$ which implies $\hat{d} = -B\nabla f(x)$ is a descent direction. ■

General formula for descent directions:

$$\boldsymbol{x}_{\boldsymbol{k}+\boldsymbol{1}} = \boldsymbol{x}_{\boldsymbol{k}} - \boldsymbol{\alpha}_{\boldsymbol{k}}\boldsymbol{B}\boldsymbol{\nabla}\boldsymbol{f}(\boldsymbol{x}_{\boldsymbol{k}}),$$
with $\boldsymbol{B} \succ \boldsymbol{0}$

Frequently used descent directions and the matrix B

- **Steepest descent**: $B_k = I \succ \mathbf{0}$
- **Newton direction**: $B_k = (\nabla^2 f(x_k))^{-1}$
- **Diagonally scaled steepest descent**: $B_k = \mathtt{diag}(d_{1,k}, \dots, d_{n,k})$
  - $d_{i,k} = \frac{\partial^2 f(x_k)}{\partial x_{ik}^2}$, for $i = 1,2, \dots, n$
- **Modified Newton direction**: $B_k = (\nabla^2 f(x_0))^{-1}$
  - Compute the hessian only at the initial point and use it thereafter
  - Or compute it at every m iterations

$$\boldsymbol{x_{k+1} = x_k - \alpha_k B \nabla f(x_k)},$$
with $\boldsymbol{B \succ 0}$

Step size (α_k) computations

- How big of a step should we take along the direction d_k

1. **Constant step size:** $\alpha_k = \alpha > 0, \ \forall k$
   - Very easy to implement
   - if α is <u>too large</u>, the iterates <u>may diverge</u>
   - If α is <u>too small</u> the iterates become <u>too slow</u>

$$\boldsymbol{x_{k+1} = x_k + \alpha_k d_k}$$

2. **Diminishing step size:** $\alpha_k \to 0$, with $\sum_{k=1}^{\infty} \alpha_k = \infty$ (e.g., $\alpha_k = \frac{1}{k}$)
   - Descent is guaranteed only when α_k becomes small → <u>slow convergence</u>
   - **Used in Deep Neural Networks**

Step size (α_k) computations

- How big of a step should we take along the direction d_k

1. **Constant step size:** $\alpha_k = \alpha > 0, \ \forall k$

2. **Diminishing step size:** $\alpha_k \to 0$, with $\sum_{k=1}^{\infty} \alpha_k = \infty$ (e.g., $\alpha_k = \frac{1}{k}$)

3. **Exact line search OR Minimization rule**: $\alpha_k = \arg\min_{\alpha \geq 0} f(x_k + \alpha d_k)$

   - This is a <u>one-dimensional optimization</u> problem

   - If f is convex function, then $f(x_k + \alpha d_k)$ is convex function w.r.t. α

4. **Inexact line search** (Armijo/Goldstein rule; see section 7.8 in [CZ13])

   - Instead of solving a one-dimensional optimization problem, it only finds an <u>approximate solution</u> for α

$$\boldsymbol{x_{k+1} = x_k + \alpha_k d_k}$$

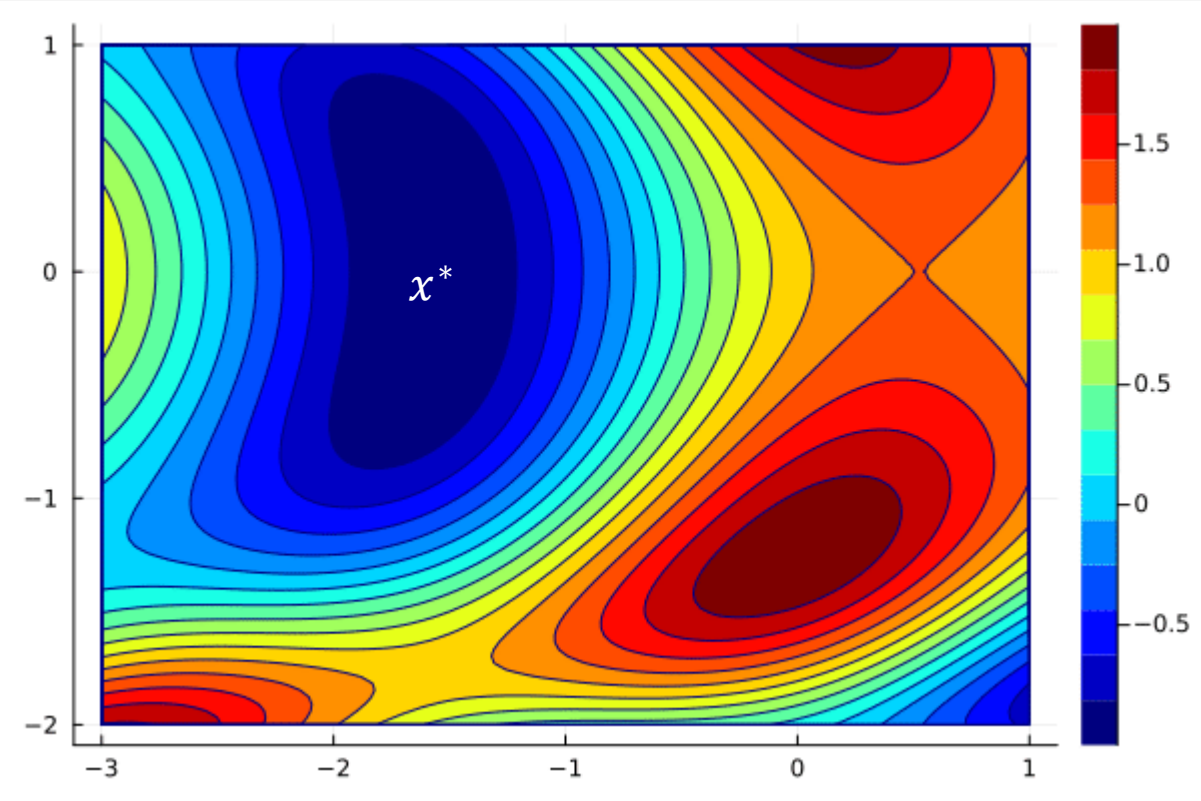
Step size (α_k) annimations

```
f(x) = sin(x[1] + x[2]^2) + cos(x[1])^2
∇f(x) = [cos(x[1] + x[2]^2) - 2*cos(x[1])*sin(x[1]); (2*x[2])*cos(x[1] + x[2]^2)]
```

$$x_{k+1} = x_k + \alpha_k \nabla f(x_k), \quad k = 1,2,\ldots,n_{iter}$$

$$\alpha_k = 0.01, \quad \forall k$$

$$n_{iter} = 100 \text{ (max \# iters)}$$



Constant **small step size;**

small but steady progress

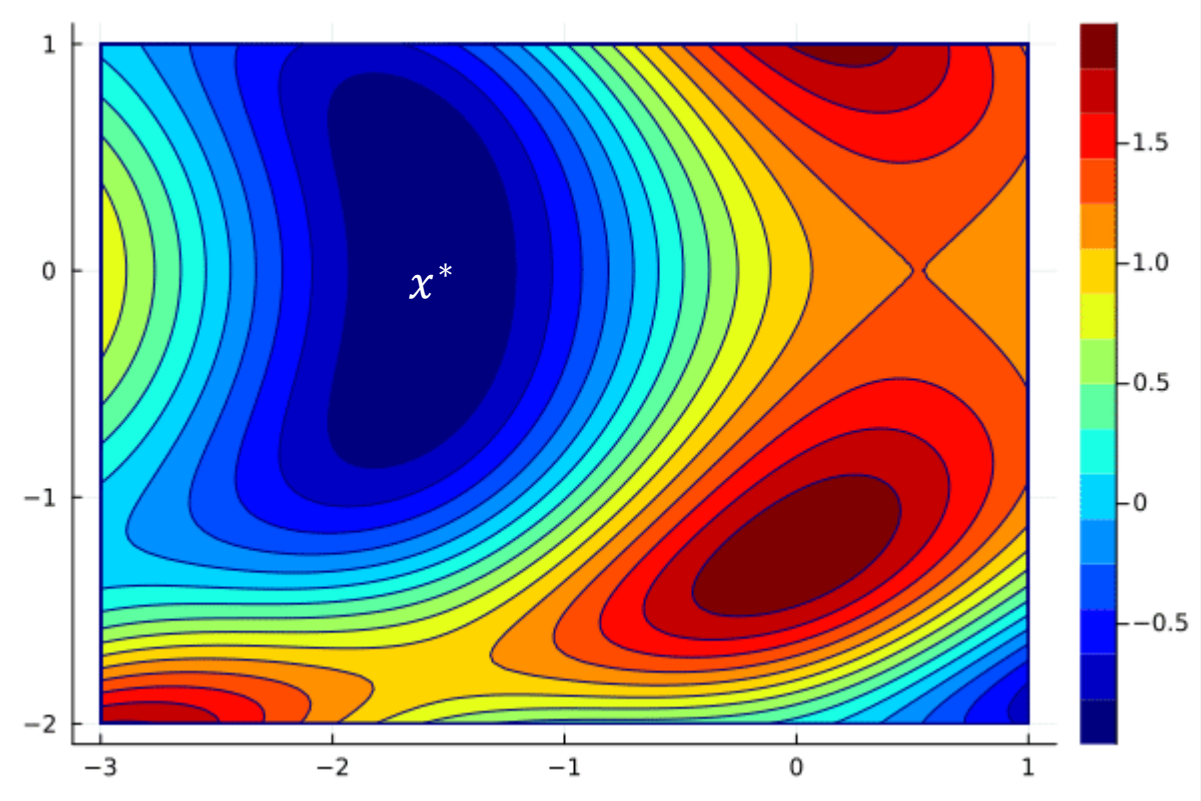
Step size (α_k) annimations

```
f(x) = sin(x[1] + x[2]^2) + cos(x[1])^2
∇f(x) = [cos(x[1] + x[2]^2) - 2*cos(x[1])*sin(x[1]); (2*x[2])*cos(x[1] + x[2]^2)]
```

$$x_{k+1} = x_k + \alpha_k \nabla f(x_k), \quad k = 1,2,\ldots,n_{iter}$$

$$\alpha_k = 0.1, \quad \forall k$$

$$n_{iter} = 100 \text{ (max \# iters)}$$



Constant **reasonable step size**;

reasonable progress

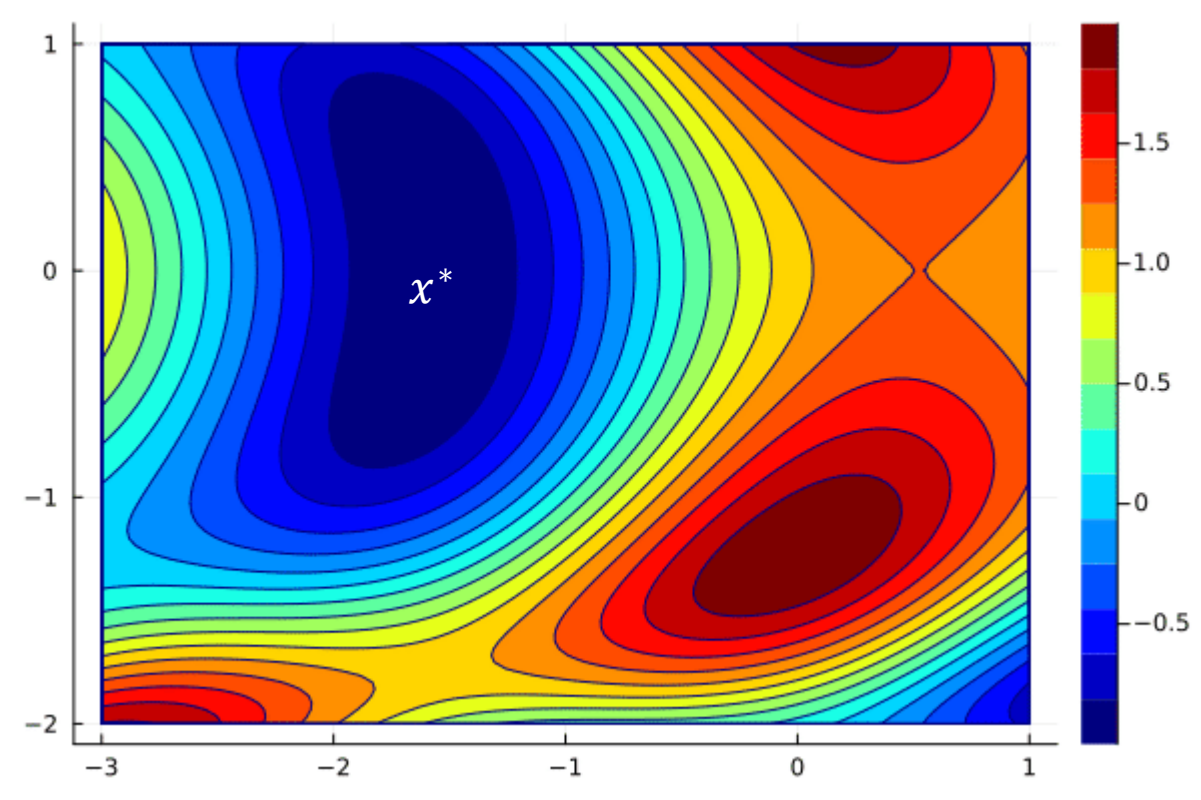
Step size (α_k) annimations

```
f(x) = sin(x[1] + x[2]^2) + cos(x[1])^2
∇f(x) = [cos(x[1] + x[2]^2) - 2*cos(x[1])*sin(x[1]); (2*x[2])*cos(x[1] + x[2]^2)]
```

$$x_{k+1} = x_k + \alpha_k \nabla f(x_k), \quad k = 1,2,\ldots,n_{iter}$$

$$\alpha_k = 1, \quad \forall k$$

$n_{iter} = 100$ (max # iters)



Constant **large step size** →

GREED IS NOT GOOD…

Stopping criteria

Every iterative method needs to know when to stop…

1. $||\nabla f(x_k)|| \leq \epsilon$ → FONC is satisfied within some tolerance $\epsilon > 0$

2. $|f(x_{k+1}) - f(x_k)| \leq \epsilon$ → improvement in the objective become very small

   - In practice we use the scaled version: $\frac{|f(x_{k+1}) - f(x_k)|}{\max\{1, |f(x_k)|\}} \leq \epsilon$

3. $||x_{k+1} - x_k|| \leq \epsilon$ → improvement in our decision variables is minimal

   - In practice we use the scaled version: $\frac{||x_{k+1} - x_k||}{\max\{1, ||x_k||\}} \leq \epsilon$

Stopping criteria

Every iterative method needs to know when to stop…

1. $||\nabla f(x_k)|| \leq \epsilon$ → FONC is satisfied within some tolerance $\epsilon > 0$

2. $|f(x_{k+1}) - f(x_k)| \leq \epsilon$ → improvement in the objective become very small

   - In practice we use the scaled version: $\frac{|f(x_{k+1})-f(x_k)|}{\max\{1,|f(x_k)|\}} \leq \epsilon$

3. $||x_{k+1} - x_k|| \leq \epsilon$ → improvement in our decision variables is minimal

   - In practice we use the scaled version: $\frac{||x_{k+1}-x_k||}{\max\{1,||x_k||\}} \leq \epsilon$

They are used for all optimization methods

Steepest descent for Quadratic functions

Let's apply the Steepest Descent method to minimize a Quadratic function:

$$\min_{x \in R^n} f(x) = \frac{1}{2} x^T Q x - b^T x \qquad \text{with } Q \succ 0$$

- The direction of movement at iteration k is: $d_k = -\nabla f(x_k)$ (known)
- The step size at iteration k is: α_k (we need to compute the <u>best step</u>)
  - We will use **exact line search** to compute α_k
  - Consider the one-variable function:

$$g(\alpha) = f\big(x_k - \alpha \nabla f(x_k)\big) = \big(x_k - \alpha \nabla f(x_k)\big)^T Q \big(x_k - \alpha \nabla f(x_k)\big) - b^T \big(x_k - \alpha \nabla f(x_k)\big)$$

Quadratic function in α

Steepest descent for Quadratic functions

$$g(\alpha) = f\big(x_k - \alpha\nabla f(x_k)\big) = \frac{1}{2}\big(x_k - \alpha\nabla f(x_k)\big)^T Q\big(x_k - \alpha\nabla f(x_k)\big) - b^T\big(x_k - \alpha\nabla f(x_k)\big).$$

Doing the **algebra** (correctly!) yields:

$$g(\alpha) = \frac{1}{2}\big(\nabla f(x_k)\big)^T Q\big(\nabla f(x_k)\big)\alpha^2 - \big(b^T - x_k^T Q\big)\nabla \mathrm{f}(\mathrm{x_k})\alpha + \left(\frac{1}{2}x_k^T Q - b^T\right)x_k.$$

Define: $c = \frac{1}{2}\big(\nabla f(x_k)\big)^T Q\big(\nabla f(x_k)\big)$, $d = \big(b^T - x_k^T Q\big)\nabla \mathrm{f}(\mathrm{x_k})$ and $e = \left(\frac{1}{2}x_k^T Q - b^T\right)x_k$

Then: $g(a) = c\alpha^2 + d\alpha + e$

$\boldsymbol{\nabla \mathrm{f}(x_k) = Qx_k - b}$

The optimal solution of $g(a)$ is obtained by: $g'(\alpha^*) = 0 \Rightarrow 2c\alpha^* + d = 0 \Rightarrow \alpha^* = -\frac{d}{2c}$

Substituting d and c yields the optimal step size: $\alpha^* = \alpha_k = -\frac{\big(b^T - x_k^T Q\big)\nabla \mathrm{f}(\mathrm{x_k})}{\big(\nabla f(x_k)\big)^T Q\big(\nabla f(x_k)\big)} = \frac{\big(\nabla f(x_k)\big)^T \nabla \mathrm{f}(\mathrm{x_k})}{\big(\nabla f(x_k)\big)^T Q\big(\nabla f(x_k)\big)}$

Steepest descent for Quadratic functions

Hence the optimal step size for Steepest Descent on quadratic functions has

the **closed form solution** $\alpha_k = -\frac{(b^T - x_k^T Q)\nabla \mathrm{f}(\mathrm{x_k})}{(\nabla f(x_k))^T Q(\nabla f(x_k))} = \frac{(\nabla f(x_k))^T \nabla \mathrm{f}(\mathrm{x_k})}{(\nabla f(x_k))^T Q(\nabla f(x_k))}$

The next iterate of steepest descent is defined by the iterative formula:

$$x_{k+1} = x_k - \underbrace{\frac{(\nabla f(x_k))^T \nabla \mathrm{f}(\mathrm{x_k})}{(\nabla f(x_k))^T Q(\nabla f(x_k))}}_{\alpha_k} \nabla f(x_k)$$

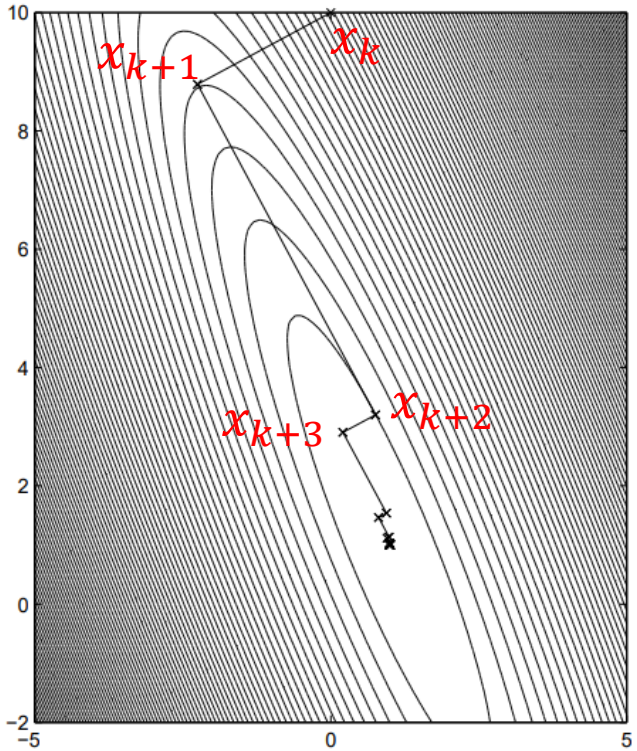
Steepest descent – the zig-zag theorem

$$f(x) = f(x_1, x_2) = \tfrac{1}{2}x^T Q x + q^T x + 20$$

$$Q = \begin{pmatrix} 10 & 4 \\ 4 & 2 \end{pmatrix} \quad \text{and} \quad q = \begin{pmatrix} -14 \\ -6 \end{pmatrix}.$$

Theorem: Let $\{x_k : k = 0,1,2,\dots\}$ be the sequence of points generated by the Steepest Descent method. Then consecutive movements generated by the algorithm are orthogonal to each other. That is, $x_{k+1} - x_k$ is orthogonal to $x_{k+2} - x_{k+1}$.

Question: What is the relationship between $\nabla f(x_k)$ and $\nabla f(x_{k+1})$??



R. Freund: "The Steepest Descent Algorithm for Unconstrained Optimization", MIT, 2014

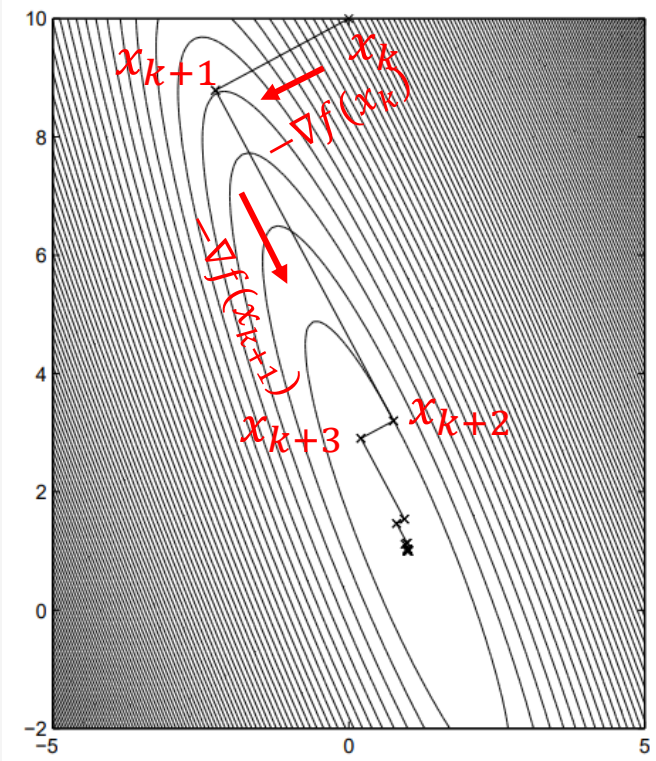
Steepest descent – the zig-zag theorem

$$f(x) = f(x_1, x_2) = \tfrac{1}{2}x^T Q x + q^T x + 20$$

$$Q = \begin{pmatrix} 10 & 4 \\ 4 & 2 \end{pmatrix} \quad \text{and} \quad q = \begin{pmatrix} -14 \\ -6 \end{pmatrix}.$$

Theorem: Let $\{x_k : k = 0,1,2,\dots\}$ be the sequence of points generated by the Steepest Descent method. Then consecutive movements generated by the algorithm are orthogonal to each other. That is, $x_{k+1} - x_k$ is orthogonal to $x_{k+2} - x_{k+1}$.

Question: What is the relationship between $\nabla f(x_k)$ and $\nabla f(x_{k+1})$??



R. Freund: "The Steepest Descent Algorithm for Unconstrained Optimization", MIT, 2014

Steepest descent – the zig-zag theorem

Proof: From the definitions of x_{k+1} and x_{k+2} we have:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k) \quad \Rightarrow \quad x_{k+1} - x_k = -\alpha_k \nabla f(x_k)$$

$$x_{k+2} = x_{k+1} - \alpha_{k+1} \nabla f(x_{k+1}) \quad \Rightarrow \quad x_{k+2} - x_{k+1} = -\alpha_{k+1} \nabla f(x_{k+1})$$

Taking the **dot product**: $(x_{k+1} - x_k)^T (x_{k+2} - x_{k+1}) = \alpha_k \alpha_{k+1} \nabla f(x_k)^T \nabla f(x_{k+1})$ **(Eq.2)**

Recall that in exact line search, α_k is the optimal solution of $\min_{\alpha} \; g(\alpha) = f(x_k - \alpha \nabla f(x_k))$

chain rule

α_k is the solution of $g'(\alpha) = 0 \Rightarrow \nabla f\big(x_k - \alpha_k \nabla f(x_k)\big)^T \big(-\nabla f(x_k)\big) = 0 \Rightarrow -\nabla f(x_{k+1})^T \nabla f(x_k) = 0$ **(Eq.3)**

Using **(Eq.2) & (Eq.3)** we have $(x_{k+1} - x_k)^T (x_{k+2} - x_{k+1}) = 0 \;\Rightarrow\; x_{k+1} - x_k$ orthogonal to $x_{k+2} - x_{k+1}$

■

Convergence of gradient descent algorithms

The descent algorithms converge to a stationary point.

Theorem: Suppose the sequence $\{x_k : k = 0,1,2, \dots\}$ is generated by the **any** descent algorithm with direction given by $d_k = -B_k \nabla f(x_k)$. Suppose also that the eigenvalues of the matrix B_k are greater than some positive number $\beta > 0$ and the step size α_k is computed by exact line search. Then $\lim_{k \to \infty} x_k = x^*$ and x^* is a stationary point of f, that is: $\nabla f(x^*) = 0$

Questions: How you would check if x^* is local minimum??

Can it be a local maximum?? What is the role of the matrix B_k ??

Convergence of gradient descent for Quadratic functions

Global convergence: If the iterates of an optimization converge to x^* from any starting point x_0

Theorem: Consider the quadratic function $f(x) = \frac{1}{2}x^T Q x + b^T x + c$ with $\boldsymbol{Q} \succ \boldsymbol{0}$ and x^* its minimizer.

- The steepest descent algorithm, with exact line search, will always converge to x^* <u>from any starting point</u> x_0 $(\lim_{k\to\infty} x_k = x^*)$
- The steepest descent algorithm with fixed step size, will converge to x^* <u>from any starting point</u> x_0 iff the step size satisfies

$$0 < \alpha < \frac{2}{\lambda_{max}(Q)}$$, where $\lambda_{max}(Q)$ is the largest eigenvalue of Q

Proof is not required

But have a look in the notes

The largest the λ_{max} the smaller the step and the slower the convergence